R4Tun: LLM-driven adaptive segmental tunnel lining segmentation in point clouds

Authors: Xinghui Tao, Guangming Wang *, Jelena Ninić, Brian Sheil

Affiliations: a: Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK b: Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, Durham, UK

Corresponding author: Guangming Wang, gw462@cam.ac.uk, Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK

Abstract

Automated inspection of segmental tunnel linings requires reliable segmentation of structural components from 3D point clouds, yet current pipelines depend on expert-tuned parameters that degrade when tunnel conditions vary. This paper presents R4Tun, a large language model (LLM) driven adaptation framework that augments an expert-designed pipeline (SAM4Tun) with bounded, context-aware parameter tuning using structured context comprising memory, state, and knowledge. Evaluated on 30 Seg2Tunnel subsets (13 regular, 17 complex tunnels) across three LLMs (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash), the full design improved mean mIoU by 108.7–118.7% overall (0.150 → 0.313–0.328), by 74.6–83.8% for regular tunnels, and by 302.4–321.4% for complex tunnels, with state contributing the largest gain. Cross-model analysis of 1,350 adapted parameter files shows all three LLMs converge on the same critical parameters and tunnel-family clustering, indicating adaptations are driven by tunnel characteristics rather than LLM biases.

Keywords: Segmental tunnel lining; Point cloud segmentation; Tunnel inspection; Large language models; Parameter adaptation; Multi-agent systems

Highlights

- Expert-tuned tunnel segmentation degrades when conditions vary from the reference configuration.
- R4Tun adds an LLM-driven adaptation layer using memory, state, and domain knowledge.
- Validated on 30 subsets covering two diameters, two ring lengths, two segment counts, and two joint types across three LLMs.
- Mean mIoU improved >100% overall, with the largest relative gains on complex tunnels (+302–321%).
- Three independent LLMs converge on the same critical parameters and adaptation patterns.

1. Introduction

Automated inspection of segmental tunnel linings is required for structural health assessment, yet reliable segmentation of lining components from 3D point clouds remains challenging due to mixed structural elements, occlusions, and noise (Attard et al., 2018; Huang et al., 2021). Three main approaches exist: feature-engineering with deterministic geometric rules that are interpretable but brittle (Duda and Hart, 1972; Fischler and Bolles, 1981); supervised deep learning that requires large labelled datasets and retraining (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023); and foundation-model pipelines like SAM4Tun (Ye et al., 2025) that achieve strong performance under expert tuning but degrade sharply when parameters are misspecified. None combines adaptability with interpretability.

In practice, a fixed expert-tuned configuration achieves mean mIoU of only 0.042 on complex tunnels compared with 0.367 on regular tunnels. This instability causes repeated expert retuning and lower confidence in automation.

We develop R4Tun, an LLM-driven adaptation framework where stage parameters are adjusted to changing tunnel conditions without expert tuning. Each LLM agent receives structured context — memory (reference characteristics), state (intermediate outputs), and knowledge (domain guidelines) — and produces bounded parameter updates via chain-of-thought reasoning.

Contributions: (1) A framework where LLM agents adapt parameters of a fixed pipeline using structured context; (2) cumulative ablation isolating each context component across 30 tunnels; (3) cross-LLM validation showing three LLMs produce consistent adaptation patterns; (4) sensitivity analysis of 1,350 parameter files identifying 11 tunnel-responsive and 7 baseline-correction parameters.

---

2. Related work

Tunnel segmentation. Feature-engineered approaches encode domain knowledge into geometric rules (Duda and Hart, 1972; Ester et al., 1996; Pauly et al., 2002) but require reconfiguration when conditions change (Weidner and Walton, 2024). Deep-learning methods reduce hand-crafting but require labelled data, training resources, and periodic retraining (Qi et al., 2017; Hu et al., 2020; Schult et al., 2023; Kolodiazhnyi et al., 2024; Cha et al., 2024).

Foundation models. SAM (Kirillov et al., 2023) enables zero-shot segmentation and has been applied to infrastructure tasks (R et al., 2024; Wang et al., 2024; Pan et al., 2023). SAM4Tun (Ye et al., 2025) combines geometric preprocessing with SAM prompting but remains highly sensitive to ~60 pipeline parameters.

LLMs as engineering agents. Reasoning-enabled LLMs can articulate intermediate steps explicitly (Wei et al., 2023; Kojima et al., 2023), and multi-agent architectures coordinate specialised roles through shared context (Hong et al., 2024; Chen et al., 2024; Liu et al., 2025). Context engineering strongly influences model performance (Xu et al., 2024; Mei et al., 2025; Anthropic, 2025). No existing approach combines LLM reasoning with expert-designed pipelines to provide adaptive, interpretable parameter tuning for infrastructure.

---

3. Materials and methods

3.1 Problem definition

The task is semantic segmentation of segmental tunnel linings from TLS point clouds: assigning each point a structural label (background, K-block, B1, B2, A1–A4). Consistent performance across varying conditions is prioritised over peak accuracy on a single configuration.

3.2 Dataset

The Seg2Tunnel benchmark comprises 30 subsets from five tunnels (Table 1).

Property	Regular (T1, T2)	Continuous (T3)	Complex (T4, T5)
Inner diameter	5.5 m	5.5 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Segments/ring	6	6	7
Joint type	Staggered	Continuous	Complex interleaved
Count	10 subsets	3 subsets	17 subsets

Regular and continuous tunnels are grouped as “regular” (n = 13); complex tunnels form a separate group (n = 17). The baseline was expert-tuned on reference tunnel T2-2 (diameter 5.60 m, density 2,466 pts/m³, mIoU 0.531).

3.3 Proposed method

R4Tun augments the five-stage SAM4Tun pipeline — Unfolding, Denoising, Enhancing, Detecting, SAM segmentation — with an LLM adaptation layer. The pipeline algorithms remain fixed; only parameter JSONs change.

Pipeline stages: (1) **Unfolding** maps the 3D point cloud to cylindrical coordinates via RANSAC centreline fitting. (2) **Denoising** removes artefacts via grid-based radial-density filtering. (3) **Enhancing** improves geometric continuity through curvature-guided upsampling. (4) **Detecting** applies Hough-transform line detection to extract ring boundaries. (5) **SAM segmentation** uses template-based prompts for segment mask generation.

LLM adaptation layer. Each stage agent receives three context components: - **Memory**: reference tunnel characteristics and expert-tuned parameters. - **State**: cumulative geometric/statistical properties from intermediate pipeline outputs (e.g., radial percentiles after unfolding, retention rates after denoising). - **Knowledge**: domain-specific parameter semantics, adaptation rules, and tunnel-family taxonomy.

The agent reasons via structured chain-of-thought: anchoring → classification → diagnostic inspection → parameter adaptation → validation. Each stage’s parameters are inferred independently from the baseline; the LLM does not see its own prior-stage outputs. The five stage scripts and evaluation code are identical across all conditions.

3.4 Experimental design

Ablation ladder. Four cumulative conditions, each adding one context component:

Level	Code	Condition	Context
0	sam4tun	Baseline	Fixed default parameters
1	m	Memory	Reference characteristics + parameters
2	m_s	Memory + State	+ intermediate pipeline outputs
3	m_s_k	Memory + State + Knowledge	+ domain knowledge

Cross-LLM validation. Each condition was run with Claude Opus 4.6, GPT-5.4, and Gemini 3 Flash under identical prompts.

Statistical testing. Paired differences Δ_i = mIoU_condition − mIoU_baseline were tested with paired t-tests ($\alpha = 0.05$), Wilcoxon signed-rank tests as non-parametric check, tunnel-level bootstrap 95% CIs (10,000 resamples), and one-sided binomial sign tests for the knowledge increment.

Evaluation metric. mIoU = (1/C) Σ IoU_c, with C = 6 (regular) or 7 (complex).

3.5 Sensitivity analysis

Three complementary analyses were conducted on 1,350 adapted parameter files (30 tunnels × 3 LLMs × 3 conditions × 5 stages): (1) **Critical parameter identification** — recording trigger frequency, cross-LLM agreement, and coefficient of variation to classify parameters as tunnel-responsive (CV ≥ 0.06) or baseline corrections (CV ≈ 0); (2) **Cross-LLM consistency** — comparing parameter keys, values, and tunnel-family clustering across models; (3) **Characteristic-to-parameter correlation** — Spearman correlations (N = 30) cross-validated against CoT text-mining.

3.6 Limitations and quality of evidence

Methodological limitations: single pipeline (SAM4Tun only); single reference configuration; SAM non-determinism (~±0.03 mIoU); single LLM run per condition; no alternative optimisation comparison; 3 continuous tunnels only.

Evidence quality summary: - *Aggregate mIoU improvement (high):* n = 30 paired design, 3 LLMs, $p < 0.0001$, Wilcoxon agreement, bootstrap CIs exclude zero. - *Component decomposition (moderate–high):* Cumulative ablation consistently ranks state > knowledge > memory across 3 LLMs. - *Cross-LLM convergence (high):* 1,350 files show convergent keys, values, and clustering. - *Memory-only effect (low):* Small, inconsistent across LLMs. - *Knowledge increment (low–moderate):* Bootstrap CIs straddle zero; but 21/30 tunnels positive for Opus/GPT (binomial $p = 0.021$); 15/17 complex positive for GPT ($p = 0.001$). - *Tunnel-bootstrap uncertainty (moderate):* Addresses benchmark composition, not SAM/LLM rerun variance.

---

4. Results

4.1 Main quantitative results

Table 2: Mean mIoU by condition and LLM (n = 30)

Condition	Opus 4.6	GPT-5.4	Gemini 3 Flash
sam4tun (baseline)	0.150	0.150	0.150
memory	0.144	0.182	0.199
memory+state	0.312	0.299	0.302
memory+state+knowledge	0.328	0.321	0.313

Table 3: Paired differences vs baseline (n = 30) with bootstrap 95% CI

Condition	Opus 4.6	GPT-5.4	Gemini 3 Flash
memory	−0.006 (p = 0.558) CI [−0.027, 0.014]	+0.032 (p = 0.028) CI [0.005, 0.058]	+0.049 (p = 0.056) CI [0.006, 0.101]
memory+state	+0.162 (p < 0.0001) CI [0.126, 0.198]	+0.149 (p < 0.0001) CI [0.110, 0.190]	+0.152 (p < 0.0001) CI [0.117, 0.189]
m+s+k	+0.178 (p < 0.0001) CI [0.139, 0.216]	+0.171 (p < 0.0001) CI [0.140, 0.203]	+0.163 (p < 0.0001) CI [0.122, 0.203]

All bootstrap CIs for m+s and m+s+k exclude zero across all LLMs.

Table 4: Mean mIoU by tunnel family (best LLM per cell)

Family	n	sam4tun	memory	memory+state	m+s+k
Regular (all)	13	0.291	0.260–0.344	0.516–0.531	0.495–0.535
Complex	17	0.042	0.055–0.101	0.126–0.155	0.169–0.177
Overall	**30**	**0.150**	**0.144–0.199**	**0.299–0.312**	**0.313–0.328**

Performance distribution (Table 4a, mean across 3 LLMs):

Metric	sam4tun	memory	memory+state	m+s+k
Mean mIoU	0.150	0.175	0.304	0.320
Std	0.166	0.136	0.228	0.218
Min	0.032	0.042	0.082	0.072
Max	0.532	0.471	0.682	0.679

Overall std increases because adaptation pulls regular tunnels to higher mIoU while complex tunnels remain lower, widening the between-family gap. Within complex tunnels, baseline std is tiny (0.003, most fail near 0.04); adaptation increases it (→ 0.074) as some tunnels recover more than others — differentiated recovery, not collapse.

Per-class IoU (Table 4b, Opus 4.6, representative): For regular tunnels, all structural classes improve roughly uniformly (doubling or more). For complex tunnels, the baseline assigns nearly all points to

background (segment IoUs = 0.000); adaptation recovers segment structure progressively. B2-block for complex tunnels remains at 0.000 under memory and memory+state and rises to only 0.028–0.095 under m+s+k — the hardest class, requiring knowledge-supplied 7-segment layout guidance.

4.2 Ablation and component analysis

Table 5: Incremental mIoU contribution (mean delta vs previous level)

Transition	Opus 4.6	GPT-5.4	Gemini 3 Flash
Baseline → Memory	−0.006	+0.032	+0.049
Memory → Memory+State	+0.168	+0.117	+0.103
Memory+State → M+S+K	+0.016	+0.022	+0.011

Memory alone is insufficient and can be harmful — it degrades regular alternated tunnels by −0.035 to −0.055. Without state feedback, the agent cannot verify whether adjustments improve intermediate outputs.

State is the dominant driver (all $p < 0.0001$; paired Cohen's $d = 1.32–1.61$, "very large"). State provides measured statistics (radial percentiles, retention rates) enabling direct parameter calibration.

Knowledge adds targeted improvement. Mean increment +0.011 to +0.022 (Cohen's $d = 0.11–0.31$, "small"). Bootstrap CIs straddle zero, but 21/30 tunnels show positive increments for Opus/GPT (binomial $p = 0.021$). Most pronounced on complex tunnels (15/17 positive for GPT, $p = 0.001$) where knowledge supplies tunnel-family-specific guidance (ring spacing, segment count, scaling factors).

4.3 Sensitivity results

Critical parameters. 18 "always-trigger" parameters identified, falling into two categories:

Tunnel-responsive (11 parameters, CV ≥ 0.06): mask_r_low/high (tunnel radius), default_cutoff_z (radial extent), z_step (scanner resolution), Hough thresholds (point density), inter_radius (point spacing), diameter (physical), processing.padding (segment width). These cluster by tunnel family — e.g., mask_r_low maps to physical radius: families 1-2 receive 2.25–2.38, families 4-5 receive 2.62–2.91.

Baseline corrections (7 parameters, CV ≈ 0): All three LLMs converge on the same corrected values (e.g., smoothing_window_size 3 → 5, curvature_threshold 0.0005 → 0.005), indicating ~7 suboptimal SAM4Tun defaults.

Cross-LLM consistency. All three LLMs adapt the same parameter keys, produce the same tunnel-family clustering, and show near-identical per-tunnel trends despite different architectures and no coordination.

Characteristic-to-parameter correlation. Strongest: unfolded r_percentiles show $|\rho| = 1.0$ with mask bounds; estimated diameter at $|\rho| \approx 0.91$ across all stages; mean nearest-neighbour distance at $|\rho| \approx 0.87$ with spacing parameters. CoT text-mining confirmed statistically significant characteristics are explicitly referenced in agent reasoning.

4.4 Error analysis

- **Memory-alone degradation** on regular tunnels (−0.035 to −0.055): agent adapts without sufficient context.
- **Continuous tunnel variability** (n = 3): high variance, poorly represented in knowledge base.
- **Complex tunnel ceiling**: absolute mIoU 0.169–0.177 despite large relative gains; compounding challenges limit parameter-only recovery.
- **Per-tunnel failures**: Tunnel 1-4 (Gemini m_s_k = 0.209, below baseline 0.348); Tunnel 4-4 (Opus m_s_k = 0.047); Tunnel 5-4 (best m_s_k = 0.122).

---

5. Discussion

5.1 Interpretation

The results support improved adaptability — the adapted pipeline achieves significantly higher mean mIoU across tunnel families that differ from the reference, consistently across three LLMs. The claim is about lifting performance on unseen configurations, not tightening the performance distribution (inter-family spread is preserved).

State dominates because it provides explicit quantitative evidence of how each stage has transformed the data. Memory alone is insufficient (a substantive finding, not a flaw): without intermediate feedback, agents make premature adjustments. Knowledge is targeted rather than uniform, most valuable for complex tunnels requiring family-specific configuration that neither memory nor state can supply.

5.2 Comparison with prior work

R4Tun differs from feature-engineered pipelines (manual reconfiguration), deep learning (labelled data + retraining), and foundation-model pipelines (parameter sensitivity) by automating adaptation while preserving interpretability. Compared with general LLM agent frameworks (Hong et al., 2024; Qian et al., 2024), R4Tun constrains agents to bounded numeric parameter changes with logged reasoning. The cross-LLM convergence provides stronger evidence than single-model evaluation.

5.3 Practical implications

For practitioners: an expert tunes a pipeline on one reference tunnel, encodes knowledge into readable documents, and deploys LLM adaptation for new tunnels. Benefits: reduced retuning effort, preserved interpretability via reasoning traces, graceful degradation (avoids baseline collapse on complex tunnels), and no retraining requirement.

5.4 Limitations

1. Single pipeline (SAM4Tun only); transferability untested.
2. Single reference; multi-dimensional deviations receive weaker anchoring.
3. No comparison with Bayesian optimisation or grid search.
4. SAM non-determinism (~±0.03 mIoU); bootstrap CIs address tunnel composition, not rerun variance.

5. LLM API dependency (cost, latency).
6. Continuous tunnel under-representation (n = 3).
7. Complex tunnel ceiling (mIoU 0.169–0.177).

5.5 Confidence assessment

Table 9: Per-claim confidence

Claim	Confidence	Supporting evidence	Key caveat
LLM adaptation raises mean mIoU	**High**	n = 30, 3 LLMs, $p < 0.0001$, bootstrap CIs exclude zero	±0.03 SAM noise not in CI
State is dominant component	**High**	Largest delta (+0.103–0.168), $d = 1.32–1.61$, all $p < 0.0001$	Interaction effects not isolated
Memory alone insufficient	**Moderate**	Cross-LLM pattern; discovery, not flaw	Rerun variance not in CI
Knowledge adds targeted improvement	**Low–Moderate**	CIs straddle zero; 21/30 positive ($p = 0.021$); 15/17 complex (GPT, $p = 0.001$)	Mean overlaps SAM noise
Adaptations driven by characteristics	**High**	3 LLMs converge on 18 parameters, Spearman $\rho \geq 0.87$	Single run per LLM
Per-class improvement is broad	**High**	All classes improve (regular); all recover from 0 (complex)	B2-block remains low
Complex tunnel ceiling exists	**High**	Best mIoU 0.169–0.177 despite +302–321% gain	May reflect pipeline limits

Overall: The central claim is supported at high confidence. Knowledge contributes a real but fragile effect (low–moderate). Remaining uncertainty: SAM/LLM rerun variance, cumulative ablation cannot isolate interactions, single pipeline and dataset.

---

6. Conclusions

R4Tun augments a fixed segmentation pipeline with LLM-driven parameter adaptation using memory, state, and knowledge. On 30 subsets across three LLMs, mean mIoU improved from 0.150 to 0.313–0.328 (bootstrap CIs exclude zero), with state contributing the largest gain (Cohen's $d > 1.3$) and all LLMs converging on the same critical parameters. Confidence in the headline improvement and state's dominance is high; confidence in the knowledge increment is low-to-moderate (Table 9). The framework reduces expert retuning while preserving transparency through logged reasoning traces. Absolute performance on complex tunnels remains modest (0.169–0.177), single-run estimates carry ~±0.03 noise, and transferability beyond SAM4Tun is untested.

Data availability

The data that support the findings of this study are available at https://github.com/Tao-Robominds/R4Tun. The Seg2Tunnel dataset is publicly available.

Declaration of competing interest

The authors declare no known competing financial interests or personal relationships.

Funding

[This work was supported by [funder] [grant number].]

Declaration of generative AI use

During the preparation of this work, the authors used large language models (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash) as the core experimental subjects of the study. The LLMs were also used for language editing and organisation of the manuscript. The authors reviewed and edited all content and take full responsibility for the content of the publication.

CRediT authorship contribution statement

[Author 1: Conceptualization, Methodology, Software, Writing – original draft.] [Author 2: Supervision, Writing – review & editing.] [Author 3: Supervision, Writing – review & editing.] [Author 4: Supervision, Funding acquisition, Project administration.]

Acknowledgements

[Acknowledge non-author support.]

References

[1] L. Attard et al. Tunnel inspection using photogrammetric techniques and image processing: A review. ISPRS J. Photogramm. Remote Sens., 144:180–188, 2018. [2] L. Weidner and G. Walton. Generalized extraction of bolts, mesh, and rock in tunnel point clouds. Remote Sens., 16(4):678, 2024. [3] M. Q. Huang, J. Ninić, and Q. B. Zhang. BIM, machine learning and computer vision techniques in underground construction. Tunn. Undergr. Space Technol., 108:103677, 2021. [4] A. Sjölander et al. Towards automated inspections of tunnels. Sensors, 23(12):5457, 2023. [5] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves. Commun. ACM, 15(1):11–15, 1972. [6] M. A. Fischler and R. C. Bolles. Random sample consensus. Commun. ACM, 24(6):381–395, 1981. [7] M. Ester et al. A density-based algorithm for discovering clusters. Proc. KDD, pp. 226–231, 1996. [8] M. Pauly, M. Gross, and L. P. Kobbelt. Efficient simplification of point-sampled surfaces. Proc. IEEE Vis., pp. 163–170, 2002. [9] C. R. Qi et al. PointNet++: Deep hierarchical feature learning on point sets. NeurIPS, 2017. [10] Q. Hu et al. RandLA-Net: Efficient semantic segmentation of large-scale point clouds. CVPR, 2020. [11] J. Schult et al. Mask3D: Mask transformer for 3D semantic instance segmentation. arXiv:2303.05475, 2023. [12] M. Kolodiazhnyi et al. Top-down beats bottom-up in 3D instance segmentation. WACV, pp. 3566–3574, 2024. [13] Y. J. Cha et al. Deep learning-based structural health monitoring. Autom. Constr., 161:105324, 2024. [14] C. Su et al. A review of deep learning applications in tunneling in China. Appl.

Sci., 14(8):3234, 2024. [15] A. Kirillov et al. Segment Anything. ICCV, pp. 3992–4003, 2023. [16] R. Bommasani et al. On the opportunities and risks of foundation models. arXiv:2108.07258, 2021. [17] R. R et al. Crack-SAM: Crack segmentation using a foundation model. arXiv:2401.15201, 2024. [18] B. Wang et al. Omni-Scan2BIM. Autom. Constr., 167:105678, 2024. [19] F. Pan et al. Zero-shot building attribute extraction. arXiv:2312.12479, 2023. [20] Z. Ye et al. SAM4Tun: No-training model for tunnel lining point cloud component segmentation. Tunn. Undergr. Space Technol., 158:106401, 2025. [21] J. Bradley et al. ChemCrow: Augmenting LLMs with chemistry tools. arXiv:2304.05376, 2023. [22] A. Ghafarollahi and M. J. Buehler. ProtAgents: Protein discovery via LLM multi-agent collaborations. Digital Discovery, 3:1956–1973, 2024. [23] C. Qian et al. ChatDev: Communicative agents for software development. arXiv:2307.07924, 2024. [24] S. Hong et al. MetaGPT: Meta programming for multi-agent collaboration. arXiv:2308.00352, 2024. [25] P. Chen et al. COMM: Collaborative multi-agent prompting. arXiv:2405.00847, 2024. [26] C. I. Garcia et al. Framework for LLM applications in manufacturing. Manuf. Lett., 40:56–63, 2024. [27] J. Wei et al. Chain-of-thought prompting elicits reasoning in LLMs. arXiv:2201.11903, 2023. [28] T. Kojima et al. Large language models are zero-shot reasoners. arXiv:2205.11916, 2023. [29] V. Xiang et al. Towards System 2 reasoning in LLMs. arXiv:2501.04682, 2025. [30] B. Liu et al. Advances and challenges in foundation agents. arXiv:2501.03428, 2025. [31] H.-A. Gao et al. A survey of self-evolving agents. arXiv:2501.02718, 2025. [32] P. Xu et al. Retrieval meets long context LLMs. arXiv:2310.03025, 2024. [33] L. Mei et al. A survey of context engineering for LLMs. arXiv:2501.04567, 2025. [34] Anthropic. Effective context engineering for AI agents. 2025. [35] OpenAI. Reasoning best practices. 2025.